Field	Details
Date	19 July 2026
Team ID	SB-AIML-2026-0042
Project Name	Emotion Detector - AI-Driven Emotion Detection & Personalized Learning Support P
Maximum Marks	3 Marks

1. Project Folder Structure

```
AI-Driven Emotion Detection & Personalized Learning Support Platform/

 streamlit_app.py              # Cloud entry point (Demo Mode)
 requirements.txt              # Cloud dependencies (lightweight)
 requirements_full.txt         # Full ML dependencies (local training)
 .env.example                  # Environment variable template
 .gitignore                    # Git ignore rules
 README.md                     # Project overview and setup guide

 app/                          # Streamlit application
    main.py                   # Local full-mode entry point
    assets/style.css          # Dark glassmorphism CSS theme
    pages/
        home.py               # Full ML detection page
        home_demo.py          # Gemini-only demo page (cloud)
        analytics.py          # Analytics dashboard page
        model_comparison.py   # BiLSTM vs BERT comparison page
        about.py              # About page

 config/
    settings.py               # All hyperparameters & paths
    emotions.py               # Emotion labels, keywords, colors

 src/                          # Core business logic
    preprocessing/            # Text cleaning & encoding
```

2. Coding Standards

Standard	Implementation
Language	Python 3.10+
Style Guide	PEP-8 compliant

Standard	Implementation
Docstrings	Google-style docstrings on all public functions
Type Hints	Used throughout `src/` modules
Line Length	Max 100 characters
Imports	Organized: stdlib third-party local
Constants	Defined in `config/settings.py` and `config/emotions.py`
Logging	Python `logging` module via `src/utils/helpers.py`

---

3. Reusability Design Patterns

Pattern	Where Used	Benefit
Singleton	`get_detector()`, `get_gemini_detector()`	Model loaded once across reruns
Strategy Pattern	`clean_for_bilstm()` vs `clean_for_bert()`	Separate preprocessing paths per model
Factory Pattern	`GuidanceGenerator.generate()`	Swappable prompt strategies per emotion
Module Isolation	Each src/ subfolder is self-contained	Can be imported/tested independently
Config-Driven	All thresholds in `settings.py`	No magic numbers in code

---

4. Sample Code Quality (Example: Detection Engine)

```python
# src/detection/ensemble.py
def ensemble_predictions(
    bilstm_probs: np.ndarray,
    bert_probs: np.ndarray,
    bilstm_weight: float = 0.40,
    bert_weight: float = 0.60,
) -> np.ndarray:
    """
    Compute weighted ensemble of BiLSTM and BERT probability vectors.

    Args:
        bilstm_probs: Softmax output from BiLSTM model, shape (5,)
        bert_probs:   Softmax output from BERT model, shape (5,)
        bilstm_weight: Weight assigned to BiLSTM (default 0.40)
        bert_weight:  Weight assigned to BERT (default 0.60)

    Returns:
        Weighted average probability vector, shape (5,)
    """
    assert abs(bilstm_weight + bert_weight - 1.0) < 1e-6, "Weights must sum to 1"
    return bilstm_weight * bilstm_probs + bert_weight * bert_probs
```

---

5. Reusability Score

Module	Can be reused independently?	Notes
`src/preprocessing/`	Yes	Generic NLP cleaning pipeline
`src/models/bilstm/`	Yes	Works for any 5-class text classification
`src/models/bert/`	Yes	Any HuggingFace text classification task
`src/detection/`	Yes	Plug in any two classifiers
`src/gemini/`	Yes	Reusable Gemini client wrapper
`src/analytics/`	Yes	Generic logger + chart library
`app/`	Partial	Depends on config/emotions.py